A-backend-contract

A — Backend Contract (Extension → Boba)

Revision: 1 **Last modified:** 2026-06-10T00:00:00Z **Authority:** Real source code (cited file:line). The browser-extension research docs (Dim01/Dim09) are NOT authoritative and were both partly wrong — see “Conflict resolution” below.

Verdict on the conflicting research claims (§11.4.123 — evidence only)

Claim	Source	Verdict	Evidence
Port **8080**, **JWT** auth, `/api/v1/torrents`	Dim09	**FALSE** — no such port, no JWT, no `/torrents` route anywhere	No `8080` in `docker-compose.yml`; no JWT/bearer-app-auth in `routes.py`/`__init__.py`/`auth.py`; the only “torrents” path is qBittorrent’s own `/api/v2/torrents/add` called server-side (`routes.py:819`)
Ports **7186/7187/7189** + `/api/v1/search|download|magnet`	Dim01	**CORRECT**	`docker-compose.yml` port env; `routes.py` route decorators;

Claim	Source	Verdict	Evidence
			`__init__.py:250` prefix

Dim01 is the accurate one. Dim09 should be discarded.

Live probe (Task 5)

Services were **DOWN** at probe time (`2026-06-10`):

```
$ curl -s -m4 http://localhost:7187/health   → (no response,
connection refused)
$ curl -s -m4 http://localhost:7187/openapi.json → (no response)
$ curl -s -m4 -o/dev/null -w '%{http_code}' http://localhost:7186/
→ 000
$ curl -s -m4 -o/dev/null -w '%{http_code}' http://localhost:7189/
→ 000
```

Services not running — endpoints derived from source (source is authoritative).

Port map (Task 4 — from `docker-compose.yml`)

Port	Service	container	docker-compose evidence	Extension uses?
7185	qBittorrent WebUI (container-internal)	`qbittorrent`	`:9` `WEBUI_PORT=7185`; health `:25`	No — answers 401 without proxy
7186	Download proxy → qBittorrent WebUI (reverse proxy, injects auth)	`qbittorrent-proxy`	`:121` `PROXY_PORT=7186`; health `:158` curls `:7186/`	Optional (raw WebUI mirror)
7187	**Merge Search Service (FastAPI default,**	`qbittorrent-proxy` / `qbittorrent-proxy-go`	`:122`/`:67` `MERGE_SERVICE_PORT=7187`; health `:104`/`:158`	**YES — primary**

Port	Service	container	docker-compose evidence	Extension uses?
	or Go/ Gin)			
7188	webui-bridge (host process / Go binary)	host / `webui-bridge`	`:70` `BRIDGE_PORT=7188`	No (private-tracker download helper)
7189	boba-jackett (Go) — Jackett creds/ overrides/ autoconfig	`boba-jackett`	`:179` `PORT=7189`; health `:195` `/healthz`	No (admin UI backend)
9117	Jackett indexer	`jackett`	`:48` health `:9117/health`	No

> Both the Python `download-proxy` container AND the Go `qbittorrent-proxy-go` container bind 7186 (proxy) + 7187 (merge) — they are mutually exclusive (Go is opt-in via `--profile go`). `main.py:88-95` runs the original proxy (7186) and FastAPI merge service (7187) as **two threads in one container**.

Direct-qBittorrent vs Boba routing (Task 3 — the key answer)

The extension talks to Boba's merge service on 7187, NOT to qBittorrent directly.

- Boba's `POST /api/v1/download` is the actual add-torrent path. Server-side it logs into qBittorrent (`{qbit_url}/api/v2/auth/login`, `routes.py:778-789`) using **admin/admin** (`_get_qbit_username/_password`, `routes.py:707-718`, default `QBITTORRENT_URL=http://localhost:7185`), then POSTs to qBittorrent's `/api/v2/torrents/add` (`routes.py:819` for tracker-fetched `.torrent`, `routes.py:847` for plain URLs/magnets via `data={"urls": url}`).
- So qBittorrent auth (admin/admin) is **handled by Boba on the server side** — the extension never sends qBittorrent credentials and never calls 7185//`api/v2`.
- Port **7186** is a separate raw reverse-proxy of the qBittorrent WebUI (the `download_proxy.run_server()` thread, `main.py:36-38`).

The extension does NOT need it for adding torrents; it exists for humans/tools wanting the WebUI.

What `/api/v1/download` actually accepts

Body model `DownloadRequest` (`routes.py:170-172`):

```
{ "result_id": "<string>", "download_urls": ["<url-or-
        magnet>", ...] }
```

`result_id` is just a label (used for the download_id/event payload, NOT a DB lookup) — there is **no requirement that the URLs came from a prior search.** The handler iterates `download_urls[:5]` (`routes.py:791`) and for each: - **Private-tracker page URL** (rutracker/kinozal/nnmclub/iptorrents host, `TRACKER_DOMAINS` `routes.py:721-731`) → fetches the real `.torrent` with auth cookies via `orch.fetch_torrent(...)`, uploads as a file (`routes.py:806-844`). - **Anything else — including an arbitrary `magnet:` link or a direct `.torrent` URL** → forwarded straight to qBittorrent as `data={"urls": url}` (`routes.py:846-861`).

=> An ad-hoc magnet IS accepted by `POST /api/v1/download` with `{"result_id":"anything","download_urls":["magnet:?xt=..."]}`. No search needed.

> **Go-profile gap:** the Go `DownloadHandler` (`internal/api/download.go:17-39`) is a **STUB** — it echoes `{"status":"added"}` for every URL **without ever contacting qBittorrent**. If the deployment runs `--profile go`, downloads silently no-op. The extension's real backend MUST be the **Python** merge service. (Flag this to the team; treat Go download path as not-implemented.)

Definitive endpoint enumeration (Task 1 — Python FastAPI, the source of truth)

Base: `http://<host>:7187`. Routers mounted in `__init__.py`: `api_router` @ `/api/v1` (`:250`), `auth_router` @ `/api/v1` (`:252`), `hooks_router` @ `/api/v1/hooks` (`:251`), `scheduler_router` @ `/api/v1/schedules` (`:253`). **No app-level auth / JWT on any route.** CORS allowlist defaults to `localhost:4200` + `localhost:7187` (`__init__.py:101-131`); extensions send `Origin: chrome-extension://…` so the team must add that origin to `ALLOWED_ORIGINS` (see Gaps).

Method	Full path	Request	Response (k
GET	`/health`	—	`{status:"heal`

Method	Full path	Request	Response (k
GET	`/api/v1/config`	—	`{qbittorrent_ proxy),qbitto`
GET	`/api/v1/stats`	—	`{active_searc`
GET	`/api/v1/bridge/health`	—	`{healthy,stat`
POST	`/api/v1/search`	`SearchRequest` (`query,category,limit,sort_by,sort_order,...`)	`SearchRespons` (returns imm
POST	`/api/v1/search/sync`	`SearchRequest`	`SearchRespons`
GET	`/api/v1/search/stream/{search_id}`	`?token=` or `Authorization: Bearer` (only if `SSE_REQUIRE_TOKEN`)	SSE `event: t` streams
GET	`/api/v1/search/{search_id}`	—	`SearchRespons`
POST	`/api/v1/search/{search_id}/abort`	—	`{search_id,st`
GET	`/api/v1/downloads/active`	—	`{downloads:[{` (queries qBit
POST	**`/api/v1/download`**	**`DownloadRequest`** `{result_id,download_urls[]}`	`{download_id,`
POST	`/api/v1/download/file`	`DownloadRequest`	streams `.tor`
POST	`/api/v1/magnet`	`{result_id,download_urls[]}`	`{magnet:"magn` magnet from
POST	`/api/v1/auth/qbittorrent`	`{username,password,save}` (defaults admin/admin)	`{status:"auth`
GET	`/api/v1/auth/status`	—	`{trackers:{ru`
GET	`/api/v1/auth/rutracker/status`	—	`{authenticate`
GET	`/api/v1/auth/rutracker/captcha`	—	`{captcha_requ`

Method	Full path	Request	Response (k
POST	`/api/v1/auth/rutracker/login`	`CaptchaLoginRequest`	`{authenticate`
POST	`/api/v1/auth/rutracker/cookie-login`	`{cookie_string}`	`{authenticate`
GET	`/api/v1/auth/nnmclub/status`	—	`{authenticate`
POST	`/api/v1/auth/nnmclub/login`	— (env creds)	`{authenticate`
POST	`/api/v1/auth/qbittorrent/logout`	—	`{status:"logg`
GET	`/api/v1/theme`, PUT `/api/v1/theme`, GET `/api/v1/theme/stream`	theme dashboard state	—
GET/POST/DELETE	`/api/v1/hooks`, `/api/v1/hooks/{id}`	webhook CRUD	—
GET/POST/DELETE	`/api/v1/schedules`, `/api/v1/schedules/{id}`	scheduled-search CRUD	—
GET	`/`, `/dashboard`, `/{path}`	Angular SPA	index.html

`SearchRequest` fields (`routes.py:110-117`)

`query` (req, min 1), `category` (def `"all"`), `limit` (1–100, def 50), `enable_metadata` (def true), `validate_trackers` (def true), `sort_by` (def `"seeds"`), `sort_order` (def `"desc"`).

Go equivalent (Task 2 — `qBitTorrent-go/`, opt-in `--profile go`)

Same ports (7186 proxy / 7187 merge / 7188 bridge), routes registered in `cmd/qbittorrent-proxy/main.go:48-94`. Endpoint set ≈ Python (search, search/sync, search/stream/:id, search/:id, search/:id/abort, download, download/file, magnet, downloads/active, auth/qbittorrent, theme, hooks, schedules, config, stats, bridge/health, health). **Differences vs Python (material):** - `DownloadHandler` is a **no-op stub** (`internal/api/download.go:17-39`) — does NOT add to qBittorrent. **Do not rely on the Go backend for downloads.** - `ActiveDownloadsHandler` returns empty `{downloads:[],count:0}` stub (`download.go:138`). - No `/api/v1/auth/rutracker/*`, `/nnmclub/*`, `/auth/status` routes (the Go proxy lacks the tracker-auth/CAPTCHA surface). - Go also has **no JWT / no 8080** — confirms Dim09 wrong on both backends.

DEFINITIVE extension → Boba integration contract (Task 6)

Base URL: **`http://<host>:7187`** (the merge service). Use the **Python** backend. No auth header required by Boba itself (no JWT).

Extension capability	Call	Body / params	Notes
(a) Add an ad-hoc magnet	`POST http://<host>:7187/api/v1/download`	`{"result_id":"ext-<id>","download_urls":["magnet:?xt=urn:btih:..."]}`	`result_id` is a free label; m forwarded to qBittorrent u (`routes.py:846`). Success = `added_count>0` / `status:"initiated"`.
(b) Add a .torrent file	`POST http://<host>:7187/api/v1/download`	`{"result_id":"ext-<id>","download_urls":["https://…/x.torrent"]}`	Direct `.torrent` URL forwa as `urls=` too. **GAP:** no mult raw-file-bytes upload endp — see Gaps.
(c) Health / auth check	`GET http://<host>:7187/health` (liveness) + `GET /api/v1/auth/status` (qBittorrent + tracker session state)	—	`/health` proves merge serv up; `/api/v1/auth/status` → `trackers.qbittorrent.has_s`
		`{"query":"...","limit":50}`	

Extension capability	Call	Body / params	Notes
(d) Search (optional)	`POST /api/v1/search/sync` (blocking, simplest) OR `POST /api/v1/search` + `GET /api/v1/search/stream/{search_id}` (live SSE)		sync returns full `results[]` `download_urls[]` to feed int (b).
Active downloads (status panel)	`GET /api/v1/downloads/active`	—	live qBittorrent torrent lis

Backend gaps the extension needs filled (Task 6 flag)

1. **CORS for the extension origin.** `ALLOWED_ORIGINS` default (`__init__.py:101-106`, docker-compose `:85`/`:137`) lists only `localhost:4200` + `localhost:7187`. A browser extension sends `Origin: chrome-extension://<id>` (or `moz-extension://…`). Either the extension must call via background `fetch` exempt from CORS (MV3 with host permissions can), OR the team adds the extension origin to `ALLOWED_ORIGINS`. `allow_credentials=False` is fine (no cookies needed). **Action: confirm the extension uses host-permission background fetch, else add the origin.**

2. **Raw .torrent file-bytes upload.** `/api/v1/download` only accepts **URLs** (`download_urls[]`). There is **no endpoint to POST raw `.torrent` file bytes** (multipart). `/api/v1/download/file` *returns* a torrent, it doesn't accept one. If the extension must add a user-picked local `.torrent` file (bytes, not a URL), Boba needs a new `POST /api/v1/download/upload` (multipart → qBittorrent `/api/v2/torrents/add` with a `torrents` form field, mirroring the server-side code already at `routes.py:811-822`). **Backend addition required.**

3. **Go-profile download is a stub.** If any deployment runs `--profile go`, `/api/v1/download` no-ops (`download.go:17-39`). Either implement the Go handler to mirror Python, or document that the extension requires the Python backend. **Backend addition required if Go is to be supported.**

4. **No JWT/token on Boba endpoints** — the extension does NOT need to obtain or send a bearer token to Boba (only the SSE stream has an optional per-search `stream_token`, off by default). Dim09's JWT requirement is fictional.